UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

INGENIERÍA EN COMPUTACIÓN

COMPUTACIÓN GRÁFICA E INTERACCIÓN HUMANO COMPUTADORA

Reporte de práctica 2: Proyecciones y puertos de vista. Transformaciones Geométricas

Nombre de la alumna: Macias Niño Carmen Violeta

Nº de Cuenta: 318285450

GRUPO DE LABORATORIO: 02

GRUPO DE TEORÍA: 02

SEMESTRE 2026-2

FECHA DE ENTREGA LÍMITE: 01/03/2026

CALIFICACIÓN: _____

INDICACIONES:

1.- Dibujar las iniciales de sus nombres en diagonal de abajo hacia arriba, cada letra de un color diferente

2.- Generar el dibujo de la casa de la clase, pero en lugar de instanciar triangulos y cuadrados será instanciando piramides y cubos, para esto se requiere crear shaders diferentes de los colores: rojo, verde, azul, café y verde oscuro en lugar de usar el shader con el color clamp. Se debe de entregar los archivos de shader diferentes dentro de un zip adicional al main y documento escrito.

DESARROLLO

1) **Includes y librerías**

```
// Práctica 2: índices, mesh, proyecciones, transformaciones geométricas
#include <stdio.h>
#include <string.h>
#include <cmath>
#include <vector>

#include <glew.h>
#include <glfw3.h>

// glm
#include <glm.hpp>
#include <gtc/matrix_transform.hpp>
#include <gtc/type_ptr.hpp>

// clases
#include "Mesh.h"
#include "Shader.h"
#include "Window.h"
```

- **stdio/string/cmath/vector:** utilidades básicas de C/C++ y contenedores (std::vector) para guardar meshes y shaders.
- **GLEW:** carga funciones modernas de OpenGL (por ejemplo, glGenBuffers, glUniformMatrix4fv, etc.). Sin GLEW muchas funciones no estarían disponibles.
- **GLFW:** crea la ventana, administra el contexto OpenGL y eventos (teclado/mouse).
- **GLM:** librería de matemáticas para OpenGL. Aquí usas:
    - glm::mat4 matrices 4x4

    - glm::translate, glm::scale para transformaciones
    - glm::perspective para proyección 3D
    - glm::value_ptr para mandar matrices al shader.
- **Mesh / Shader / Window:** tus clases del "código base":
    - Window inicializa la ventana y buffers
    - Shader carga/compila/linkea shaders y da getModelLocation(), getProjectLocation()
    - Mesh maneja VAO/VBO/EBO y RenderMesh().

2) Variables globales y rutas de shaders

```
const float toRadians = 3.14159265f / 180.0f;

Window mainWindow;
std::vector<Mesh*> meshList;
std::vector<Shader> shaderList;

// Shaders por color (vertex cambia el color, fragment es común)
static const char* fShaderSolid = "shaders/shaderColorSolid.frag";
static const char* vShaderRojo = "shaders/shaderrojo.vert";
static const char* vShaderVerde = "shaders/shaderverde.vert";
static const char* vShaderAzul = "shaders/shaderazul.vert";
static const char* vShaderCafe = "shaders/shadercafe.vert";
static const char* vShaderVerdeOscuro = "shaders/shaderverdeoscuro.vert";
```

- toRadians sirve para convertir grados a radianes (por si luego rotas con grados).
- mainWindow es tu ventana principal.
- meshList guarda punteros a objetos geométricos (piramide y cubo).
    - Te permite instanciar varias veces el **mismo mesh** cambiando solo su matriz model (traslación/escala).
- shaderList guarda shaders ya compilados.
- Las rutas "shaders/..." indican que los archivos .vert y .frag se cargarán desde una carpeta shaders que debe existir **junto al ejecutable** o según tu *working directory*.
- La idea del profe: **ya no usas color clamp**, sino **un shader por color** (rojo, verde, azul, café, verde oscuro).

3) CreaPiramide(): geometría con índices (EBO)

```cpp
// ======== Geometrías ========

// Pirámide triangular regular (índices)
void CreaPiramide()
{
    unsigned int indices[] = {
        0, 1, 2,
        1, 3, 2,
        3, 0, 2,
        1, 0, 3
    };

    GLfloat vertices[] = {
        -0.5f, -0.5f,  0.0f,   // 0
         0.5f, -0.5f,  0.0f,   // 1
         0.0f,  0.5f, -0.25f,  // 2 (punta)
         0.0f, -0.5f, -0.5f    // 3
    };

    Mesh* piramide = new Mesh();
    piramide->CreateMesh(vertices, indices, 12, 12);
    meshList.push_back(piramide); // meshList[0]
}
```

- Crea una pirámide triangular con 4 vértices:
    - base “triangular” (o mejor dicho, una base formada por triángulos según índices)
    - vértice/punta es el índice 2.
- indices[] define **cómo se conectan** esos vértices para formar triángulos:
    - cada grupo de 3 índices es un triángulo.
    - esto cumple con “índices” de la práctica: usas EBO/Index Buffer en CreateMesh.
- vertices[] solo tiene posición (x, y, z). No hay color por vértice.
- CreateMesh(vertices, indices, 12, 12):
    - 12 vértices “floats” totales (4 * 3)
    - 12 índices totales.
- Se guarda en meshList[0] para renderizarla muchas veces (techo, copa del árbol).

4) CrearCubo(): cubo con índices (36 índices)

```
}

// Cubo (indices)
void CrearCubo()
{
    unsigned int cubo_indices[] = {
        // front
        0, 1, 2,
        2, 3, 0,
        // right
        1, 5, 6,
        6, 2, 1,
        // back
        7, 6, 5,
        5, 4, 7,
        // left
        4, 0, 3,
        3, 7, 4,
        // bottom
        4, 5, 1,
        1, 0, 4,
        // top
        3, 2, 6,
        6, 7, 3
    };

    GLfloat cubo_vertices[] = {
        // front
        -0.5f, -0.5f,  0.5f,
         0.5f, -0.5f,  0.5f,
         0.5f,  0.5f,  0.5f,
        -0.5f,  0.5f,  0.5f,
        // back
        -0.5f, -0.5f, -0.5f,
         0.5f, -0.5f, -0.5f,
         0.5f,  0.5f, -0.5f,
        -0.5f,  0.5f, -0.5f
    };

    Mesh* cubo = new Mesh();
    cubo->CreateMesh(cubo_vertices, cubo_indices, 24, 36);
    meshList.push_back(cubo); // meshList[1]
}
```

- Un cubo tiene 6 caras. Cada cara son 2 triángulos → 12 triángulos total.
- 12 triángulos * 3 vértices = **36 índices**.
- Solo usas 8 vértices (las esquinas del cubo).

  Los índices dicen cómo formar las caras con esos vértices.
- 24 floats en vertices: (8 * 3).
- Se guarda en meshList[1]:
  - paredes, puerta, ventanas, pasto, tronco y hasta las letras (pixel art) se hacen con cubos instanciados.

5) CreateShaders(): un shader por color sólido

```
// ======== Shaders ========

void CreateShaders()
{
    Shader sR;  sR.CreateFromFiles(vShaderRojo, fShaderSolid);            shaderList.push_back(sR);  // [0] rojo
    Shader sG;  sG.CreateFromFiles(vShaderVerde, fShaderSolid);           shaderList.push_back(sG);  // [1] verde
    Shader sB;  sB.CreateFromFiles(vShaderAzul, fShaderSolid);            shaderList.push_back(sB);  // [2] azul
    Shader sC;  sC.CreateFromFiles(vShaderCafe, fShaderSolid);            shaderList.push_back(sC);  // [3] café
    Shader sDO; sDO.CreateFromFiles(vShaderVerdeOscuro, fShaderSolid);    shaderList.push_back(sDO); // [4] verde oscuro
}
```

Qué hace en detalle:

- Carga y compila 5 programas shader distintos:
    - Todos comparten el MISMO fragment shader (shaderColorSolid.frag).
    - Cambia el vertex shader, porque ahí se fija el color vColor = vec4(...).
- Esto es exactamente lo que te pidieron:
    - “crear shaders diferentes de los colores: rojo, verde, azul, café y verde oscuro”.

Por qué funciona así:

- Como tu geometría NO trae colores por vértice (solo posiciones), el color se decide en el shader.
- vColor sale del vertex shader y llega al fragment shader.
- En fragment shader se pinta ese color en pantalla.

6) RenderMeshInstanced(): el corazón de “instanciar” (aunque sea manual)

```cpp
// ======== Helpers de render ========

void RenderMeshInstanced(Mesh* m,
    Shader& sh,
    const glm::vec3& pos,
    const glm::vec3& scale,
    GLuint& uniformModel,
    GLuint& uniformProjection,
    const glm::mat4& projection)
{
    sh.useShader();
    uniformModel = sh.getModelLocation();
    uniformProjection = sh.getProjectLocation();

    glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));

    glm::mat4 model(1.0f);
    model = glm::translate(model, pos);
    model = glm::scale(model, scale);

    glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
    m->RenderMesh();
}
```

1. sh.useShader() activa el programa shader para dibujar lo siguiente.
2. getModelLocation() y getProjectLocation() obtienen el “handle” de los uniforms model y projection (variables uniform en el shader).
3. glUniformMatrix4fv(uniformProjection, ...) manda la proyección al shader.
4. Construye model:
    - inicia identidad (no hace nada)
    - translate mueve el objeto a pos
    - scale cambia su tamaño a scale
5. glUniformMatrix4fv(uniformModel, ...) manda esa matriz al shader.
6. m->RenderMesh() finalmente dibuja el mesh.

- Con una sola geometría (un cubo y una pirámide) creas muchos objetos:
    - casa completa y letras

7) Patrones LETTER_C/V/M/N: “pixel art” 5×7

```
// ======== Letras CVMN como “pixel art” (5x7) ========

static const char* LETTER_C[7] = {
    "11111",
    "10000",
    "10000",
    "10000",
    "10000",
    "10000",
    "11111"
};

static const char* LETTER_V[7] = {
    "10001",
    "10001",
    "10001",
    "10001",
    "01010",
    "01010",
    "00100"
};
```

```
static const char* LETTER_M[7] = {
    "10001",
    "11011",
    "10101",
    "10001",
    "10001",
    "10001",
    "10001"
};

static const char* LETTER_N[7] = {
    "10001",
    "11001",
    "10101",
    "10011",
    "10001",
    "10001",
    "10001"
};
```

- Cada arreglo tiene 7 filas (alto).
- Cada string tiene 5 caracteres (ancho).
- ‘1’ significa “aquí hay un cubo”.
- ‘0’ significa “vacío”.

8) RenderLetterCubes(): convierte el patrón en cubos en el espacio

```
void RenderLetterCubes(const char* pattern[7],
    const glm::vec3& origin,
    float cubeSize,
    Shader& shaderColor,
    GLuint& uniformModel,
    GLuint& uniformProjection,
    const glm::mat4& projection)
{
    shaderColor.useShader();
    uniformModel = shaderColor.getModelLocation();
    uniformProjection = shaderColor.getProjectLocation();
    glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));

    // pattern: 7 filas, 5 columnas
    for (int row = 0; row < 7; row++)
    {
        for (int col = 0; col < 5; col++)
        {
            if (pattern[row][col] == '1')
            {
                glm::mat4 model(1.0f);
                float x = origin.x + col * cubeSize;
                float y = origin.y + (6 - row) * cubeSize; // invertimos para que suba
                float z = origin.z;

                model = glm::translate(model, glm::vec3(x, y, z));
                model = glm::scale(model, glm::vec3(cubeSize, cubeSize, cubeSize));

                glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
                meshList[1]->RenderMesh(); // cubo
            }
        }
    }
}
```

- Activa el shader del color (por eso cada letra puede tener color distinto).
- Recorre el patrón 5×7.
- Cuando encuentra un ‘1’:
    - calcula posición:
        - x = origin.x + col * cubeSize → desplaza a la derecha por columnas
        - y = origin.y + (6-row) * cubeSize → invierte filas para que “arriba” se vea arriba (en vez de al revés)
        - z = origin.z → misma profundidad para todos los cubitos
    - escala cada cubo al tamaño cubeSize.
    - dibuja el cubo meshList[1].

9) RenderCasa(): composición de la casa con cubos y pirámides

```
// ======== Dibujo Casa 3D (cubos + pirámides) ========

void RenderCasa(const glm::vec3& basePos,
    GLuint& uniformModel,
    GLuint& uniformProjection,
    const glm::mat4& projection)
{
    // indices de shaderList:
    // 0 rojo, 1 verde, 2 azul, 3 café, 4 verde oscuro

    // Pared principal (cubo café)
    RenderMeshInstanced(meshList[1], shaderList[3],
        basePos + glm::vec3(0.0f, 0.0f, 0.0f),
        glm::vec3(2.2f, 1.6f, 1.6f),
        uniformModel, uniformProjection, projection);
```

```
// Techo (2 pirámides rojas para "cubrir" el ancho)
RenderMeshInstanced(meshList[0], shaderList[0],
    basePos + glm::vec3(-0.65f, 1.35f, 0.0f),
    glm::vec3(1.6f, 1.25f, 1.6f),
    uniformModel, uniformProjection, projection);

RenderMeshInstanced(meshList[0], shaderList[0],
    basePos + glm::vec3(0.65f, 1.35f, 0.0f),
    glm::vec3(1.6f, 1.25f, 1.6f),
    uniformModel, uniformProjection, projection);

// Pasto (cubo verde oscuro) como base del suelo
RenderMeshInstanced(meshList[1], shaderList[4],
    basePos + glm::vec3(0.0f, -1.35f, 0.0f),
    glm::vec3(5.0f, 0.2f, 3.0f),
    uniformModel, uniformProjection, projection);
```

```
// Puerta (cubo azul)
RenderMeshInstanced(meshList[1], shaderList[2],
    basePos + glm::vec3(0.0f, -0.38f, 0.85f),
    glm::vec3(0.55f, 0.85f, 0.12f),

    uniformModel, uniformProjection, projection);

// Ventanas (cubos verdes)
RenderMeshInstanced(meshList[1], shaderList[1],
    basePos + glm::vec3(-0.75f, 0.1f, 0.85f),
    glm::vec3(0.45f, 0.45f, 0.12f),
    uniformModel, uniformProjection, projection);

RenderMeshInstanced(meshList[1], shaderList[1],
    basePos + glm::vec3(0.75f, 0.1f, 0.85f),
    glm::vec3(0.45f, 0.45f, 0.12f),
    uniformModel, uniformProjection, projection);
```

```
        uniformModel, uniformProjection, projection);

    // Árbol: tronco café + copa verde oscuro (pirámide)
    RenderMeshInstanced(meshList[1], shaderList[3],
        basePos + glm::vec3(2.6f, -0.75f, 0.0f),
        glm::vec3(0.35f, 1.0f, 0.35f),
        uniformModel, uniformProjection, projection);

    RenderMeshInstanced(meshList[0], shaderList[4],
        basePos + glm::vec3(2.6f, 0.35f, 0.0f),
        glm::vec3(1.05f, 1.05f, 1.05f),
        uniformModel, uniformProjection, projection);
}
```

- meshList[1] = cubo
- shaderList[3] = café
- basePos + vec3(...) define la posición relativa a la casa
- scale vec3(...) define tamaño de esa parte.

Elementos:

- **Pared**: cubo café escalado grande.
- **Puerta**: cubo azul más delgado (Z pequeña), centrado al frente.
- **Ventanas**: cubos verdes delgados, a los lados.
- **Techo**: dos pirámides rojas para cubrir el ancho.

- **Pasto**: cubo verde oscuro ancho y plano (Y muy chica).
- **Árbol**: tronco café (cubo alto) + copa verde oscuro (pirámide).

Por qué basePos sirve mucho:

- Mueves TODA la casa con una sola línea en el main:
    - RenderCasa(glm::vec3(2.0f, -1.1f, -11.0f), ...)
- Así la mandas a la esquina y no tapa letras.

10) main(): inicialización + loop de render

```
int main()
{
    mainWindow = Window(800, 800);
    mainWindow.Initialise();

    glEnable(GL_DEPTH_TEST);

    // Geometrías
    CreaPiramide();  // meshList[0]
    CrearCubo();     // meshList[1]

    // Shaders colores
    CreateShaders();

    // Depth
    glEnable(GL_DEPTH_TEST);

    GLuint uniformProjection = 0;
    GLuint uniformModel = 0;

    // Proyección perspectiva
    glm::mat4 projection = glm::perspective(
        glm::radians(60.0f),
        (float)mainWindow.getBufferWidth() / (float)mainWindow.getBufferHeight(),
        0.1f, 100.0f
    );
```

- Crea ventana y contexto OpenGL.
- glEnable(GL_DEPTH_TEST) activa Z-buffer:
    - objetos más cerca tapan a los de atrás correctamente.
- Crea geometrías (piramide y cubo).
- Carga shaders.

Proyección perspectiva

glm::mat4 projection = glm::perspective(glm::radians(60.0f),
width/height, 0.1f, 100.0f);

Qué hace:

- Define cámara tipo 3D:
  - 60° FOV
  - aspecto = ancho/alto
  - near=0.1, far=100
- Esto cumple el punto de "proyecciones".

Loop

```
while (!mainWindow.getShouldClose())
{
    glfwPollEvents();

    glClearColor(0.0f, 0.0f, 0.0f, 1.0f);
    glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);

    // ======= 1) Letras CVMN en diagonal (abajo -> arriba) =======
    glm::vec3 base(-2.2f, -1.6f, -7.0f);
    float s = 0.12f; // tamaño cubito

    // C rojo
    RenderLetterCubes(LETTER_C, base, s, shaderList[0], uniformModel, uniformProjection, projection);

    // V verde (sube y se mueve a la derecha)
    RenderLetterCubes(LETTER_V, base + glm::vec3(1.1f, 1.1f, 0.0f), s, shaderList[1], uniformModel, uniformProjection, projection);

    // M azul
    RenderLetterCubes(LETTER_M, base + glm::vec3(2.2f, 2.2f, 0.0f), s, shaderList[2], uniformModel, uniformProjection, projection);


    // N café
    RenderLetterCubes(LETTER_N, base + glm::vec3(3.3f, 3.3f, 0.0f), s, shaderList[3], uniformModel, uniformProjection, projection);

    // ======= 2) Casa (cubos + pirámides) =======

    RenderCasa(glm::vec3(2.0f, -1.1f, -11.0f), uniformModel, uniformProjection, projection);

    glUseProgram(0);
    mainWindow.swapBuffers();
}

return 0;
```

- Poll de eventos: teclado/mouse.
- Limpia pantalla y profundidad cada frame.
- Dibuja letras CVMN en diagonal:
  - cada letra tiene su origin desplazado.

    - cada una con shader distinto = color distinto.
- Dibuja casa en esquina inferior derecha usando basePos.
- Intercambia buffers para mostrar lo renderizado.

RESULTADO